```
"""
HR Predictor Pro - Backend API
==============================
Serves home run probability predictions by combining:
  1. A logistic regression model trained on Statcast-style mock data
     (exit velocity, launch angle, stadium factor, wind speed -> HR or not)
  2. REAL live season stats pulled from the official public MLB Stats API
     (https://statsapi.mlb.com) - no API key required.

NOTE ON DATA HONESTY:
The MLB Stats API gives season-level box stats (HR count, AVG, ERA, WHIP, etc.)
for free. It does NOT give you per-pitch Statcast metrics (actual exit velocity /
launch angle for a specific at-bat) without a paid/partner feed. So this backend
*derives* plausible exit-velocity/launch-angle inputs from a player's real season
power numbers, runs them through the trained model, and is transparent about that
in the API response via the "data_source" field. This is a reasonable, defensible
approach for a hobby/demo app - just don't market it as official Statcast data.
"""

from flask import Flask, request, jsonify
from flask_cors import CORS
import requests
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

app = Flask(__name__)
CORS(app)

MLB_API_BASE = "https://statsapi.mlb.com/api/v1"
CURRENT_SEASON = 2026

# ============================================================================
# 1. CORE ML SETUP
# ============================================================================
# Mock Statcast-style training data: [exit_velo, launch_angle, stadium_factor, wi
historical_at_bats = np.array([
    [105.0, 28.0, 1.10,  5.0, 1],
    [92.0,  15.0, 1.00,  0.0, 0],
    [110.0, 35.0, 0.95, -5.0, 1],
    [98.0,  50.0, 1.05,  2.0, 0],
    [102.0, 26.0, 0.85,  0.0, 0],
    [99.0,  24.0, 1.20, 12.0, 1],
    [85.0,  10.0, 1.00,  8.0, 0],
```

```
    [107.0, 30.0, 1.00,  1.0, 1],
    [95.0,  22.0, 1.00,  0.0, 0],
    [108.0, 29.0, 1.05,  3.0, 1],
    [90.0,  18.0, 0.90, -3.0, 0],
    [112.0, 32.0, 1.15,  6.0, 1],
])
X_train = historical_at_bats[:, 0:4]
y_train = historical_at_bats[:, 4]

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
model = LogisticRegression()
model.fit(X_train_scaled, y_train)


# ====================================================================
# 2. LIVE MLB STATS API HELPERS (real, free, public endpoints)
# ====================================================================
def get_player_season_stats(person_id, group):
    """
    Fetch real current-season stats for a player from the public MLB Stats API.
    group must be 'hitting' or 'pitching'.
    Returns a dict of stats, or None if unavailable (e.g. player hasn't played ye
    """
    url = f"{MLB_API_BASE}/people/{person_id}/stats"
    params = {"stats": "season", "group": group, "season": CURRENT_SEASON}

    try:
        resp = requests.get(url, params=params, timeout=5)
        resp.raise_for_status()
        data = resp.json()
        splits = data.get("stats", [{}])[0].get("splits", [])
        if not splits:
            return None
        return splits[0].get("stat", {})
    except (requests.RequestException, KeyError, IndexError, ValueError):
        return None


def get_player_info(person_id):
    """Fetch basic bio info (name, current team) for a player."""
    url = f"{MLB_API_BASE}/people/{person_id}"
    try:
        resp = requests.get(url, timeout=5)
        resp.raise_for_status()
        people = resp.json().get("people", [])
        return people[0] if people else None
```

```
    except (requests.RequestException, KeyError, IndexError, ValueError):
        return None


def derive_exit_velocity(season_hrs, season_avg):
    """
    Derive a plausible average exit velocity from real season power numbers.
    This is a heuristic, not measured Statcast data - documented clearly.
    League average exit velo is ~88-89 mph; elite power hitters trend higher.
    """
    base = 88.0
    hr_boost = min(season_hrs, 50) * 0.35
    avg_boost = max(season_avg - 0.250, 0) * 40
    return round(base + hr_boost + avg_boost, 1)


def derive_launch_angle(season_hrs):
    """Heuristic launch angle derived from power profile. League avg ~12-14deg;
    HR-friendly swings trend toward 25-30deg."""
    base = 14.0
    boost = min(season_hrs, 50) * 0.28
    return round(min(base + boost, 32.0), 1)


def derive_pitcher_vulnerability(era, whip):
    """
    Converts pitcher ERA/WHIP into a 'stadium_factor'-style multiplier the model
    can use as a proxy for how hittable the pitcher has been this season.
    Lower ERA/WHIP = tougher pitcher = lower multiplier.
    """
    era = era if era and era > 0 else 4.50
    whip = whip if whip and whip > 0 else 1.30
    factor = 0.75 + (era / 12.0) + (whip - 1.0) * 0.3
    return round(max(0.7, min(factor, 1.4)), 3)


# ============================================================================
# 3. API ENDPOINTS
# ============================================================================
@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"})


@app.route("/predict", methods=["POST"])
def predict():
    """Manual prediction using raw Statcast-style inputs (no live API call)."""
```

```
    try:
        data = request.get_json()
        exit_velocity = float(data.get("exit_velocity", 100.0))
        launch_angle = float(data.get("launch_angle", 25.0))
        stadium_factor = float(data.get("stadium_factor", 1.0))
        wind_speed = float(data.get("wind_speed", 0.0))

        input_data = np.array([[exit_velocity, launch_angle, stadium_factor, wind
        input_scaled = scaler.transform(input_data)
        probability = model.predict_proba(input_scaled)[0][1] * 100

        return jsonify({"status": "success", "probability": round(probability, 1)
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 400


@app.route("/predict_live", methods=["POST"])
def predict_live():
    """
    Real matchup prediction: pulls actual current-season MLB stats for the given
    batter_id and pitcher_id, derives model inputs, and returns a probability.
    """
    try:
        req = request.get_json()
        batter_id = req.get("batter_id")
        pitcher_id = req.get("pitcher_id")
        wind_speed = float(req.get("wind_speed", 0.0))
        stadium_factor_override = req.get("stadium_factor")

        if not batter_id or not pitcher_id:
            return jsonify({"status": "error", "message": "batter_id and pitcher_

        batter_stats = get_player_season_stats(batter_id, "hitting")
        pitcher_stats = get_player_season_stats(pitcher_id, "pitching")

        data_source = "live_mlb_stats_api"
        if batter_stats is None:
            batter_stats = {"homeRuns": 15, "avg": ".250"}
            data_source = "fallback_defaults"
        if pitcher_stats is None:
            pitcher_stats = {"era": "4.20", "whip": "1.30"}
            data_source = "fallback_defaults" if data_source == "live_mlb_stats_a

        season_hrs = int(batter_stats.get("homeRuns", 15))
        season_avg = float(batter_stats.get("avg", ".250"))
        era = float(pitcher_stats.get("era", "4.20"))
        whip = float(pitcher_stats.get("whip", "1.30"))
```

```
        live_ev = derive_exit_velocity(season_hrs, season_avg)
        live_la = derive_launch_angle(season_hrs)
        live_stadium = float(stadium_factor_override) if stadium_factor_override

        input_data = np.array([[live_ev, live_la, live_stadium, wind_speed]])
        input_scaled = scaler.transform(input_data)
        probability = model.predict_proba(input_scaled)[0][1] * 100

        return jsonify({
            "status": "success",
            "probability": round(probability, 1),
            "data_source": data_source,
            "debug_metrics": {
                "derived_exit_velocity": live_ev,
                "derived_launch_angle": live_la,
                "pitcher_vulnerability_factor": live_stadium,
                "batter_season_hrs": season_hrs,
                "batter_season_avg": season_avg,
                "pitcher_season_era": era,
                "pitcher_season_whip": whip,
            }
        })
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 400


@app.route("/player_search", methods=["GET"])
def player_search():
    """
    Search for a player by name using the real MLB Stats API.
    Lets the frontend look up any current player instead of a hardcoded list.
    """
    name = request.args.get("name", "").strip()
    if not name:
        return jsonify({"status": "error", "message": "name query param required"

    url = f"{MLB_API_BASE}/people/search"
    try:
        resp = requests.get(url, params={"names": name}, timeout=5)
        resp.raise_for_status()
        people = resp.json().get("people", [])
        results = [
            {
                "id": p.get("id"),
                "name": p.get("fullName"),
                "position": p.get("primaryPosition", {}).get("abbreviation"),
```

```
                "team": p.get("currentTeam", {}).get("name", "N/A"),
            }
            for p in people
        ]
        return jsonify({"status": "success", "results": results})
    except requests.RequestException as e:
        return jsonify({"status": "error", "message": str(e)}), 502


if __name__ == "__main__":
    print("--- HR Predictor Pro backend starting on port 5001 ---")
    app.run(host="0.0.0.0", port=5001, debug=True)
```